

SPECTROview: An Open-Source Application for Interactive Spectroscopic Data Processing, Fitting, and Visualization

Van Hoan Le ^{a,*}

^a Univ. Grenoble Alpes, CEA, LETI, Grenoble 38000, France

* Corresponding author. E-mail address: van-hoan.le@cea.fr

Abstract

SPECTROview is an open-source, Python-based desktop application for interactive spectroscopic data processing, multi-peak fitting, and visualization. It was developed primarily for semiconductor material characterization and metrology using Raman spectroscopy, FTIR and photoluminescence, and the examples presented in this paper reflect this application domain. However, its core functionalities, including data import, preprocessing (cropping and baseline correction), peak fitting, and visualization, are domain-independent and can be applied to spectroscopic data from a wide range of fields, including biomedical spectroscopy, chemistry, and geosciences. The software is organized into three integrated workspaces (Spectra, Maps, and Graphs) that cover the complete analysis workflow, from importing discrete spectra or hyperspectral data to interactive preprocessing, multi-peak fitting, and publication-quality statistical visualization. A key contribution is the Vectorized Batch Fit (VBF) engine, a batched Levenberg-Marquardt optimizer that fits all spectra of a dataset simultaneously using vectorized NumPy operations and analytical Jacobians, achieving up to 60× speed up over conventional *lmfit* based per-spectrum approaches. The software is released under the GPL-3.0 license and is freely available on GitHub and PyPI.

Keywords

VBF engine; Spectroscopic data processing; Hyperspectral data; Peak fitting; Data visualization

Metadata

Nr	Code metadata description	Metadata
C1	Current code version	26.26.2
C2	Permanent link to code/repository used for this code version	https://github.com/CEA-MetroCarac/SPECTROview
C3	Legal code license	GNU General Public License (GPL-3.0)
C4	Code versioning system used	git
C5	Software code languages, tools and services used	Python
C6	Compilation requirements, operating environments and dependencies	Python between (3.8 & 3.12); PySide6, SciPy, Pandas, Matplotlib, Seaborn (cf. pyproject.toml for the complete list).
C7	If available, link to developer documentation/manual	https://CEA-MetroCarac.github.io/SPECTROview/
C8	Support email for questions	van-hoan.le@cea.fr

1. Motivation and significance

Spectroscopic techniques such as Raman scattering, FTIR and photoluminescence are widely used across various application fields including materials science, chemistry, biology, geology, and semiconductor manufacturing. Modern spectrometers can acquire either individual spectra or hyperspectral datasets, in which a complete spectrum is recorded at each point of a two-dimensional grid, yielding maps containing thousands to tens of thousands of spectra. Extracting meaningful insights from raw spectroscopic datasets usually requires a multi-step workflow encompassing data import, spectral pre-processing (e.g., spectral range selection, cropping, and baseline subtraction), multiple-peak fitting, aggregation of best-fit results and statistical visualization. In practice, this workflow is often fragmented across multiple software solutions. Data acquisition is typically performed using vendor-specific proprietary software, such as Renishaw WiRE or Horiba LabSpec, which generally offers limited flexibility for peak fitting, requires tedious manual result extraction and aggregation, provides minimal support for batch processing and custom statistical visualization, and is subject to restrictive licensing.

General-purpose open-source tools such as fityk [1] provide flexible curve fitting with parameter constraints but lack native support for hyperspectral data visualization and processing. Library RamanSPy [2] offer powerful preprocessing and machine-learning pipelines but lack a graphical user interface (GUI) for interactive peak fitting. Similarly, PyFasma [3] offers a Jupyter notebook-based API for Raman preprocessing with spectral deconvolution using Imfit package [4]. However, it also lacks a GUI, native hyperspectral map visualization, or batch-fitting capabilities. More recent python-based tools such as fitspy [5] (which also relies on Imfit library [4] for fitting features) support analysis of both individual spectra and hyperspectral datasets. However, their performance can degrade significantly for medium- and large-scale datasets because the fitting workflow remains largely based on sequential Python-level iteration. While multithreading and multiprocessing can provide partial acceleration, the Global Interpreter Lock (GIL) limits true parallel execution of CPU-bound tasks, and multiprocessing incurs additional overhead from data serialization and inter-process communication. As a result, both scalability and computational efficiency remain challenging on standard multi-core workstations.

Beyond peak-fitting-oriented tools, several software solutions have been proposed that address complementary aspects of spectroscopic data analysis, with an emphasis on chemometric modelling and spectral classification. ParLeS [6] provides a GUI-based environment for multivariate calibration and prediction including PCA and partial least-squares regression (PLSR) and was designed primarily for soil science and geoscience spectroscopic data, though it lacks support for multi-peak fitting or hyperspectral map visualization. Reisner et al. [7] developed an integrated, cross-platform system for Raman spectra processing that combines preprocessing, PCA, and machine-learning classification (LDA, SVM, and ANN) within a GUI environment, targeting spectral identification tasks rather than quantitative peak fitting. More recently, RAMANMETRIX [8] introduced a web-based platform covering the full pipeline from Raman spectral preprocessing to chemometric and deep-learning model training, with applications in clinical and industrial diagnostics. Similarly, RAMBO [9] is an open-source web application for Raman spectral preprocessing and biological tissue characterization, offering automated peak detection and molecular band assignment via a curated spectral database. For Raman hyperspectral mapping specifically, De Souza et al. [10] proposed an open-access Python-based

scripting routine that applies classical least squares (CLS), PCA, and PLS methods to preprocess mapping datasets and generate chemical distribution maps, though it operates without a graphical interface. While these tools address important aspects of spectroscopic data analysis particularly chemometric decomposition and spectral classification none integrates interactive multi-peak fitting with parameter constraints, high-performance batch fitting for large hyperspectral datasets, and publication-quality statistical visualization within a single GUI application.

To the best of our knowledge, no existing free and open-source software package combines: (i) support for multiple spectroscopic data formats, (ii) interactive spectral preprocessing (e.g., cropping and baseline subtraction) and multi-peak fitting with parameter constraints, (iii) native handling of both discrete spectra and hyperspectral datasets, including 2D maps and semiconductor wafer maps, (iv) high-performance batch fitting optimized for large-scale datasets, and (v) integrated statistical analysis and publication-quality visualization within a single application.

SPECTROview was developed at the Platform for Nano-characterisation (PFNC) of CEA-Leti to address this gap. Designed by researcher-engineers for researcher-engineers, it streamlines the complete spectroscopic analysis workflow from raw data import to visualization within a unified and user-friendly environment. Its feature set is driven by the practical challenges of processing the large volumes of spectroscopic data generated daily in both academic research laboratories and semiconductor manufacturing environments. A key innovation of SPECTROview application is the Vectorized Batch Fit (VBF) engine, a high-performance implementation of the Levenberg-Marquardt algorithm specifically designed for large-scale hyperspectral datasets. Although batched optimization is widely employed in modern deep-learning frameworks, most open-source spectroscopy software still performs fitting sequentially, optimizing spectra one at a time. In contrast, the VBF engine reformulates the fitting of N spectra as a single tensor-based linear algebra problem implemented entirely in NumPy. Importantly, this vectorized approach preserves the independent optimization of each spectrum by maintaining spectrum-specific convergence states. A dynamic convergence mask progressively excludes converged spectra from subsequent iterations, thereby minimizing unnecessary computations throughout the fitting process. By combining the computational efficiency of batched numerical operations with the robustness of independent Levenberg-Marquardt optimization, the VBF engine achieves speedups of up to 60× compared with conventional sequential spectrum-by-spectrum fitting approaches. This performance improvement enables the routine analysis of large hyperspectral datasets on standard workstations, substantially reducing processing times while maintaining fitting accuracy.

2. Software description

2.1. Software architecture

SPECTROview is a Python-based application utilizing PySide6 (Qt 6) [11] for its graphical user interface (GUI). The application follows a strict Model-View-ViewModel (MVVM) architecture that separates data management (Model), application logic and orchestration (ViewModel), and user interface layout (View). This modular design enables independent testing of the numerical core and simplifies future extensions of the user interface. Figure 1 illustrates the architecture and module interactions.

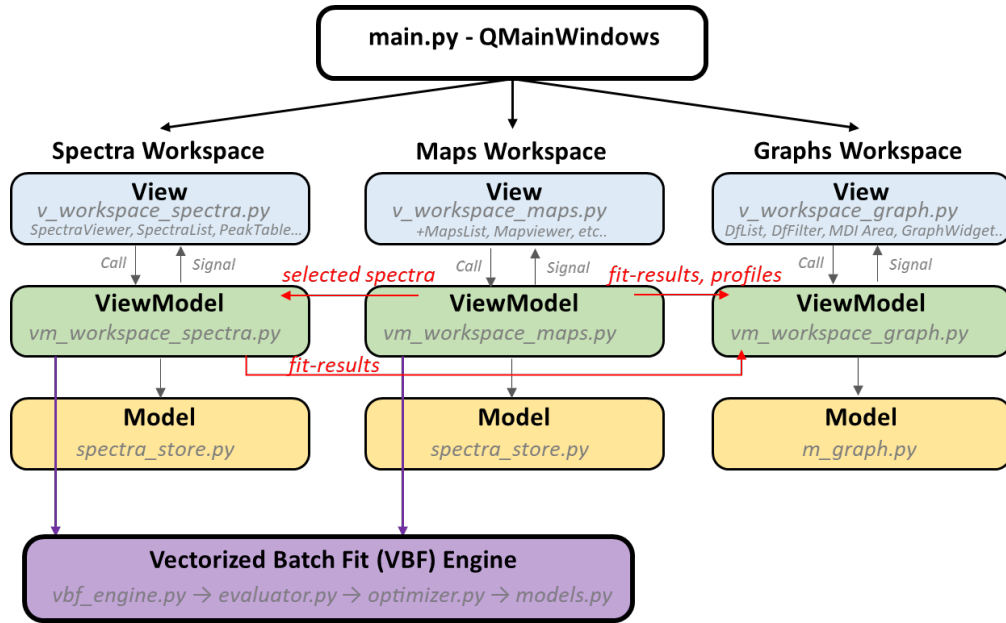


Figure 1: Simplified architecture of SPECTROview illustrating the three main workspaces (Spectra, Maps, and Graphs), their Model–View–ViewModel (MVVM) layers, and the communication between workspaces. The architecture separates the graphical user interface from the data-processing logic, facilitating modular development and maintenance.

To ensure a seamless workflow, cross-workspace communications (indicated by red arrows shown in Figure 1) are managed via Qt signal-slot connections and startup dependency injection. Key workflow examples include:

- Transferring best-fit results from the Spectra or Maps Workspace directly to the Graphs Workspace for immediate statistical analysis and visualization.
- Sending individual spectra of a given 2Dmap to the Spectra Workspace for detailed inspection and inter-2Dmap comparison, bypassing the need for file-based exports.
- Exporting line profiles (extracted via two-point selection on a 2D heatmap plot) directly to the Graphs Workspace, transferring both the raw data and the resulting plot.

A comprehensive architectural description is provided in the developer guide, available at: <https://cea-metrocarac.github.io/SPECTROview/developer/>.

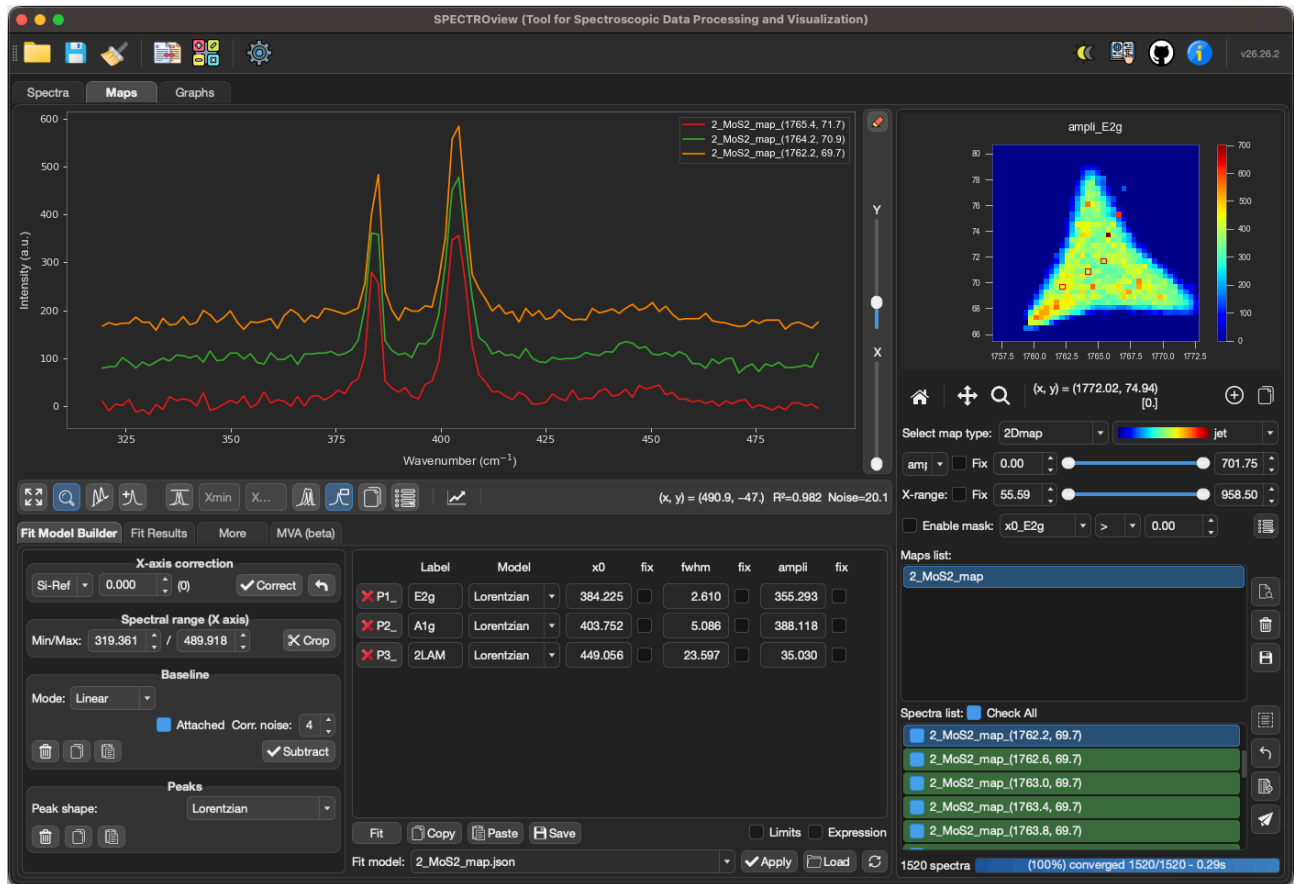


Figure 2: Main graphical user interface of SPECTROview, organized into three workspaces: Spectra, Maps, and Graphs. Only one workspace is visible at a time; users switch between them by selecting the corresponding tabs at the top left of the main window. In this example, the **Maps** workspace is active, displaying a 2D map plot, visualization controls, and map analysis tools. An animated version of the interface is available in the [online documentation](#).

The application's user interface is organized into three workspaces, each implemented as a tab within a QTableWidget. Users can switch between workspaces by selecting the corresponding tabs at the top left of the main window. Figure 2 and Figure 6 illustrate the Maps and Graphs workspaces, respectively.

- Spectra Workspace: manages discrete spectra including loading, preprocessing, interactive multi-peak fitting, and best fit result collection.
- Maps Workspace: inherits and extends the Spectra Workspace to handle hyperspectral datasets. It adds spatial heatmap visualization (2D maps and wafer maps) and includes dedicated list boxes to navigate between multiples loaded datasets and individual spectra within them.
- Graphs Workspace: provides a dataframe-based statistical plotting environment with support for eight plot styles and dynamic filtering. This allows users to work on a single datasheet across multiple plots.

A comprehensive description of the GUI and its functionalities of each workspace is provided in the User Manual, available at: https://cea-metrocarac.github.io/SPECTROview/user_manual/. The core software functionalities are selectively and briefly introduced in Section 2.2.

2.2. Software functionalities

2.2.1 Data import

Table 1 summarizes all file formats currently supported by SPECTROview for importing spectroscopic and tabular data used for analysis and visualization. Example datasets for each supported format are provided in [the examples folder](#) of the GitHub repository. The import system is readily extensible, and support for additional file formats can be added by extending the *m_io.py* module.

Table 1: File formats supported by SPECTROview.

Format	Extension	Loader	Data type
Delimited text	.txt, .csv	Auto-detected delimiter	Single spectra and maps
Renishaw WiRE	.wdf	RenishawWiRE library	Single spectra and maps
SPC	.spc	Built-in binary reader	Single spectra and maps
TRPL	.dat	Built-in parser	Time-resolved decays
Tabular data	.xlsx, .csv	Pandas & Openpyxl	DataFrames for plotting

2.2.2 Preprocessing

The preprocessing pipeline operates on working copies of the raw data, preserving the original arrays for non-destructive analysis. Available operations include:

- X-axis correction (optional): linear shift of the x-axis.
- Spectral range selection: interactive cropping of the x-axis range.
- Baseline correction: both manual (linear, polynomial with user-placed anchor points) and automatic (arPLS, airPLS, asLS, etc.) baseline subtraction. Automatic baseline algorithms are provided by the pybaselines package [12] which leverages optimized NumPy/SciPy solvers [13], and can be previewed in real time before subtraction.

These operations are fully vectorized, allowing users to apply preprocessing steps to an individual spectrum or universally across all loaded maps.

2.2.3 Multiple peaks fitting with the VBF engine

Users interactively define peak models and select peaks by clicking directly on the spectrum. Each peak is assigned a line shape and populated into a peak table (as shown in Figure 2), where initial parameters are auto estimated and users can define bounds or inter-parameter constraints.

Fitting relies on the VBF engine, a custom batched Levenberg-Marquardt optimizer [14]. Rather than iterating over spectra in a Python loop, the VBF engine reformulates the simultaneous fitting of N spectra into coupled tensor operations:

- The N observed spectra are stacked into an $(N \times M)$ data matrix, where M is the number of spectral points.
- Initial parameter guesses are assembled into an $(N \times K)$ matrix, where K is the number of free parameters, with per-spectrum amplitude scaling.
- Model evaluation and Jacobian computation produce $(N \times M)$ and $(N \times M \times K)$ tensors, respectively.
- Matrix multiplications and transpositions for the normal equations ($J^T J$ and $J^T r$) are assembled and performed via *np.einsum*, and solved in a single LAPACK-dispatched *np.linalg.solve* call [15], yielding the parameter updates for all spectra simultaneously.

All built-in peak models utilize analytical Jacobians, avoiding the $2K$ extra function evaluations per iteration required by finite-difference methods. Despite batching, spectra still converge independently: the optimizer maintains a per-spectrum damping factor and a boolean convergence mask that progressively excludes converged spectra from subsequent iterations.

Additional performance optimizations include: (i) a template-based model application that pre-computes Model objects once and clones only lightweight parameter dictionaries, (ii) batch preprocessing that computes range masks and baseline indices once for spectra sharing the same x-axis, (iii) an adaptive solver that selects between batched *np.linalg.solve* [10] and Cholesky decomposition (*cho_solve*) [10] based on matrix dimensions, and (iv) noise-aware weight masking that ignores sub-noise-floor data to prevent ghost peaks.

Entire fit models including baseline settings, peak definitions, and parameter constraints can be copied, and pasted across loaded datasets, or can be saved as reusable analysis templates.

Currently, the VBF engine supports eight built-in peak line shapes, including Gaussian, Lorentzian, Pseudo-Voigt, asymmetric Gaussian, asymmetric Lorentzian, Fano, single-exponential decay, and bi-exponential decay. Each model is implemented with an analytical Jacobian to maximize computational efficiency. Additional details are provided in the [User Manual](#) and [Developer Guide](#). Different line shapes can be freely combined within the same fitting model. The fitting engine imposes no hard-coded limit on the number of fitted peaks. However, computation time increases with the number of free parameters. The current implementation has been successfully validated using models containing more than 10 peaks per spectrum. The software is readily extensible, and support for additional peak models can be implemented as needed. Users are welcome to contribute new line shapes or submit feature requests through the [project's GitHub repository](#).

2.2.4 Hyperspectral map visualization

The Maps Workspace renders loaded hyperspectral datasets as interactive heatmaps (Figure 2). The Z-axis (color scale) can display raw integrated intensity, integrated area, or any best fit results (cf. section 2.2.5).

Users can interact with the heatmap to select individual or multiple spectra (via simple mouse click or hold Ctrl + clicking on the heatmap), define regional subsets using a rectangular drag selection, or extract spatial line profiles between two points. The selected spectra are immediately displayed in the plot widget for inspection and fitting, while extracted line profiles can be exported directly to the Graphs Workspace as DataFrames.

SPECTROview also supports wafer-map rendering using *scipy.griddata* interpolating the data onto a regular grid constrained by a circular mask.

2.2.5 Best-fit results aggregation and export

Once curve fitting has been performed on a set of individual spectra (Spectra workspace) or an entire hyperspectral dataset (Maps workspace), SPECTROview provides an integrated interface for compiling, transforming, and exporting the fitting results.

In the Fit Results tab, clicking Collect Fit Results automatically gathers all best-fit parameters (e.g., peak position, amplitude, and FWHM) into a sortable Pandas DataFrame for interactive visualization (Figure 3).

A built-in filename parser can automatically extract metadata from filenames (e.g., sample identifiers, process conditions, or measurement temperatures, etc.) and store them as additional DataFrame columns, eliminating the need for manual annotation (Figure 3, annotation n°2). This feature is highly advantageous for users employing systematic file naming conventions during data acquisition.

	Filename	X	Y	Quadrant	Zone	x0_1	x0_2	x0_3
1	wafer4_process1	0.0	0.0	nan	Center	392.686	414.72	383.311
2	wafer4_process1	29.7	0.0	nan	Center	392.648	414.963	384.477
3	wafer4_process1	21.0	20.978	Q3	Center	392.73	414.866	383.935
4	wafer4_process1	0.0	29.667	nan	Center	393.041	414.561	384.893
5	wafer4_process1	-21.0	20.978	Q2	Center	392.926	414.482	383.935
6	wafer4_process1	-29.7	0.0	nan	Center	392.715	414.447	383.626
7	wafer4_process1	-21.0	-20.977	Q1	Center	392.536	414.573	383.241
8	wafer4_process1	0.0	-29.667	nan	Center	392.671	414.655	383.463
9	wafer4_process1	21.0	-20.977	Q4	Center	392.736	414.802	384.213
10	wafer4_process1	59.3	0.0	nan	Mid-Radius	392.859	415.232	384.351
11	wafer4_process1	54.8	22.706	Q3	Mid-Radius	393.085	415.25	384.141

Figure 3: **Fit Results** tab used to compile, analyze, and export fitting results from multiple spectra. The left panel provides tools to (1) collect fitting results, (2) extract metadata from filenames, (3) create computed columns using mathematical expressions, (4) transfer the resulting table to the Graphs workspace for further analysis, and (5) export the results to Excel or CSV files. The right panel displays a table containing the best-fit results for all processed spectra. Additional details and an animated demonstration are available in the [“Collect and Save Fit Results”](#) section of the User Manual.

Users can also create computed columns by applying mathematical expressions to existing fit parameters using the standard operators (+, −, ×, ÷, **). Figure 3 (annotation n°3) illustrates this feature with the expression $(ampli_1 / ampli_2) \times 2$, in which the amplitude of Peak 1 is divided by that of Peak 2 and the result is then multiplied by two.

The resulting DataFrame can be (i) transferred directly to the Graphs Workspace for immediate statistical visualization (Figure 3, annotation n°4), or (ii) exported as an Excel (`.xlsx`) or CSV (`.csv`) file (Figure 3, annotation n°5). By integrating result compilation, metadata extraction, data transformation, and visualization within a single workflow, SPECTROview eliminates the intermediate export/import steps typically required by many spectroscopy analysis tools, thereby accelerating the generation of publication-ready results.

2.2.5 Statistical plotting

The Graphs Workspace operates on Pandas DataFrames loaded either from supported files (CSV, excel) or transferred from other workspaces. It supports various plot types: *point*, *scatter*, *box*, *bar*, *line*, *trendline*, *histogram*, *wafer*, and *2Dmap* plots. Visualizations are rendered using *Seaborn* [16] and *Matplotlib* [17] within a Multiple Document Interface (MDI, Qt) windows, enabling the simultaneous display and comparison of multiple plots (Figure 6). A graph customization dialog provides axis limits, annotations, and export to clipboard or image file.

For wafer datasets containing a *Slot* column, a batch wafer-plotting mode automatically generates multiple selected wafer plots at once. In addition, the intelligent “collect fit results” feature can automatically identify data originating from different wafer regions, such as quadrants or radial zones (center, mid-radius, and edge), based on the wafer diameter and XY coordinates of spectra (an example is shown in results data table in Figure 3). This functionality simplifies statistical analysis and comparison across multiple wafers and wafer’s regions using a single dataset, without requiring additional data cleaning or post-processing.

2.2.6 Multivariate analysis

SPECTROview includes built-in multivariate analysis capabilities: Principal Component Analysis (PCA) via singular value decomposition and Non-negative Matrix Factorization (NMF) via Lee-Seung’s multiplicative update rules [18]. Both methods operate on preprocessed spectra and produce scree plots, loadings, and score visualizations. Results can be exported to the Graphs Workspace for further analysis.

2.2.7 Save and resumes works

SPECTROview uses dedicated native file formats to persist the complete analysis state of each workspace. Clicking the “Save” button in the menu bar generates a workspace-specific archive:

- *.spectra* : stores all loaded 1D spectra, baseline configurations, fit models, and aggregated fit results.
- *.maps* : stores the raw 2D/wafer map data, instrument metadata, associated spectra, and point-by-point fit results.
- *.graphs* : stores all loaded DataFrames, plot configurations, and annotations.

To guarantee the rapid serialization of large datasets, the Spectra and Maps workspaces archives are ZIP-compressed, bundling JSON metadata with compressed NumPy arrays. The Graphs workspace archives utilizes a gzip-compressed JSON format.

Users can resume a session by opening or dragging and dropping a saved workspace file (*.spectra*, *.maps*, *.graphs*), directly into the main window. SPECTROview auto-detects the format,

navigates to the appropriate workspace, and restores the exact analysis state. This ensures strict reproducibility and seamless collaboration.

2.2.8 Other Utilities

In addition to its three core workspaces, SPECTROview integrates some auxiliary modules accessible from the main menu bar:

- [Quick Calculators](#) (Appendix A): A set of interactive scientific calculators designed for routine spectroscopy use in the laboratory. These include calculations for laser spot size and depth of focus, as well as optical penetration depth estimates based on laser wavelength and extinction coefficient, with direct access to the refractiveindex.info database and accompanying pedagogical illustrations. This module also includes a Unit Converter, enabling bidirectional conversions between wavelength (nm), energy (eV), and wavenumber (cm^{-1}), as well as Raman shift calculations for a given laser excitation wavelength.
- Hyperspectral Data Converter: A tool for converting hyperspectral text files into formats supported by SPECTROview.

These utilities are implemented as lightweight, self-contained dialog modules that can be invoked at any point during an analysis session without interrupting the current workspace state.

3. Illustrative examples

This section demonstrates a representative workflow using example datasets distributed with SPECTROview in the *examples/* folder of the GitHub repository. An intuitive User Manual with animated demonstrations of various functionalities, is integrated in the application (accessible via User Manual button in the main GUI menu bar). An online version is also available at https://cea-metrocarac.github.io/SPECTROview/user_manual/.

3.1. Fitting discrete spectra in Spectra workspace

Once the spectra are loaded, the user can process them through a sequence of steps: (i) defining a spectral region of interest, (ii) performing baseline correction using either manual or automatic modes, (iii) defining one or more peaks, and (iv) executing the fit.

Peaks are placed interactively by clicking directly on the spectrum plot. Each peak can be further configured by adjusting its line shape, parameter bounds, or inter-parameter expressions. Once all peaks are defined, clicking "Fit" dispatches the data to the VBF engine, which fits the selected spectra simultaneously. The fitted curves, individual peak contributions, and residuals are then displayed in the spectrum viewer. Users can also load a predefined fit model from a previous session and click the "Apply" button to execute the entire fitting pipeline in a single action.

Figure 4 presents an example of a fitted MoS_2 spectrum with multiple peaks, exported directly from the application. The user can adjust several parameters before exporting, such as the figure's aspect ratio, peak labels, grid lines, legend entries, and individual spectrum colors. A comprehensive list of available view options is detailed in the User Manual.

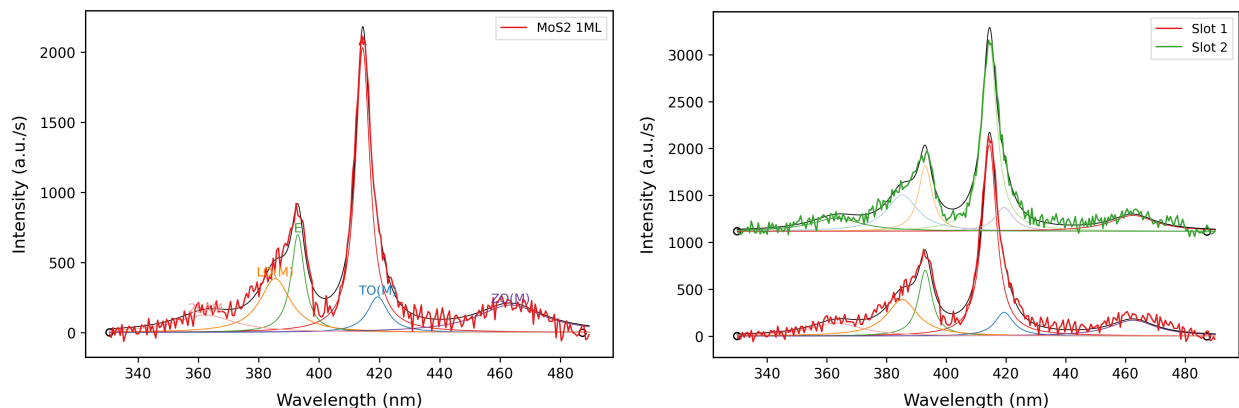


Figure 4: (a) MoS₂ Raman spectrum after baseline correction and six-component peak fitting. The raw spectrum (red), individual peak components (colored), and overall fitted curve (black) are displayed. User-defined peak labels can be shown or hidden to improve readability. (b) Simultaneous visualization of multiple selected spectra with an adjustable vertical offset, enabling detailed inspection and comparison.

3.2. Analyzing a hyperspectral map in Maps Workspace

As described in Section 2.1, the Maps Workspace inherits all core features of the Spectra Workspace, with adaptations to the user interface and processing logic designed to facilitate navigation between maps and between individual spectra within a given map. Once loaded, hyperspectral map files are listed in the MapList widget, and all individual spectra from the selected map populate the SpectraList widget (right panel, Figure 2). By default, the heatmap viewer displays the integrated intensity across the selected spectral range. The fitting workflow remains identical to that used in the Spectra Workspace (Section 3.1).

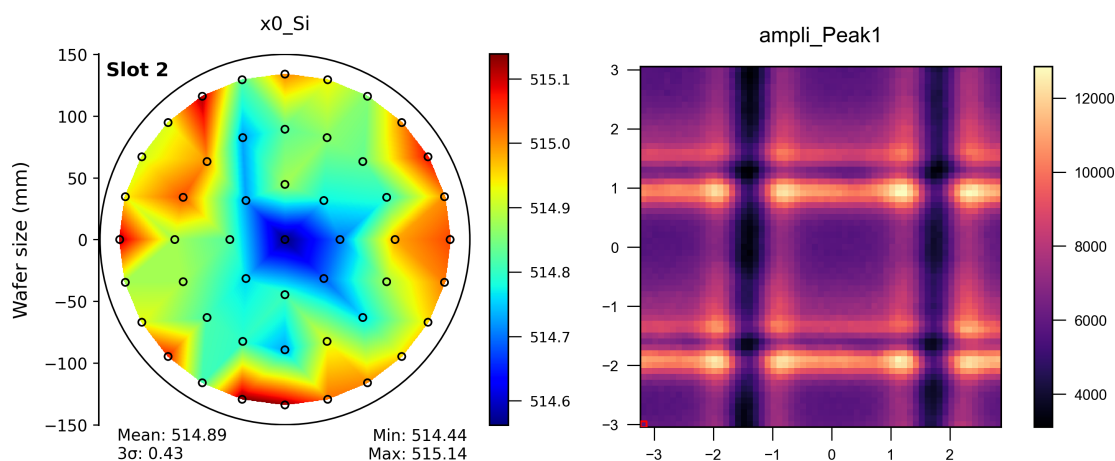


Figure 5: (a) Wafer map of a 300 mm silicon wafer showing the spatial distribution of the fitted Raman peak position (color scale in cm⁻¹). Each circular marker represents an individual spectral acquisition point on the wafer surface. Statistical summaries (mean, minimum, maximum, and standard deviation) of the displayed parameter are automatically calculated and shown alongside the map. (b) Two-dimensional intensity map of the fitted intensity of Peak 1, illustrating spatial variations across the measured area.

The VBF engine processes the entire map (or all loaded maps if hold “Ctrl” key and clicking) as a single batch operation. Following the fit, the Z-parameter dropdown updates to include the newly

fitted parameters (e.g., peak position, FWHM, amplitude). The heatmap is then re-rendered to visualize the spatial distribution of each fitted parameter across all defined peaks (Figure 5).

3.3. Plotting graphs

Figure 6 present the GUI of the Graphs workspace. Data for visualization can be loaded from external CSV for Excel files or imported directly from the best-fit results forwarded from the Spectra or Maps workspaces (Figure 3). Multiple datasets can be loaded and displayed in the list box located in the right-side panel (Figure 6). The user selects a dataset from the list panel, assigns columns to the X, Y, and optionally Z axes via dropdown menus, and chooses a plot style (e.g., scatter, box, bar, line, trendline, histogram, wafer, or 2D map). Clicking "Add Plot" renders the figure in the MDI area. Multiple plots can be displayed simultaneously for side-by-side comparison.

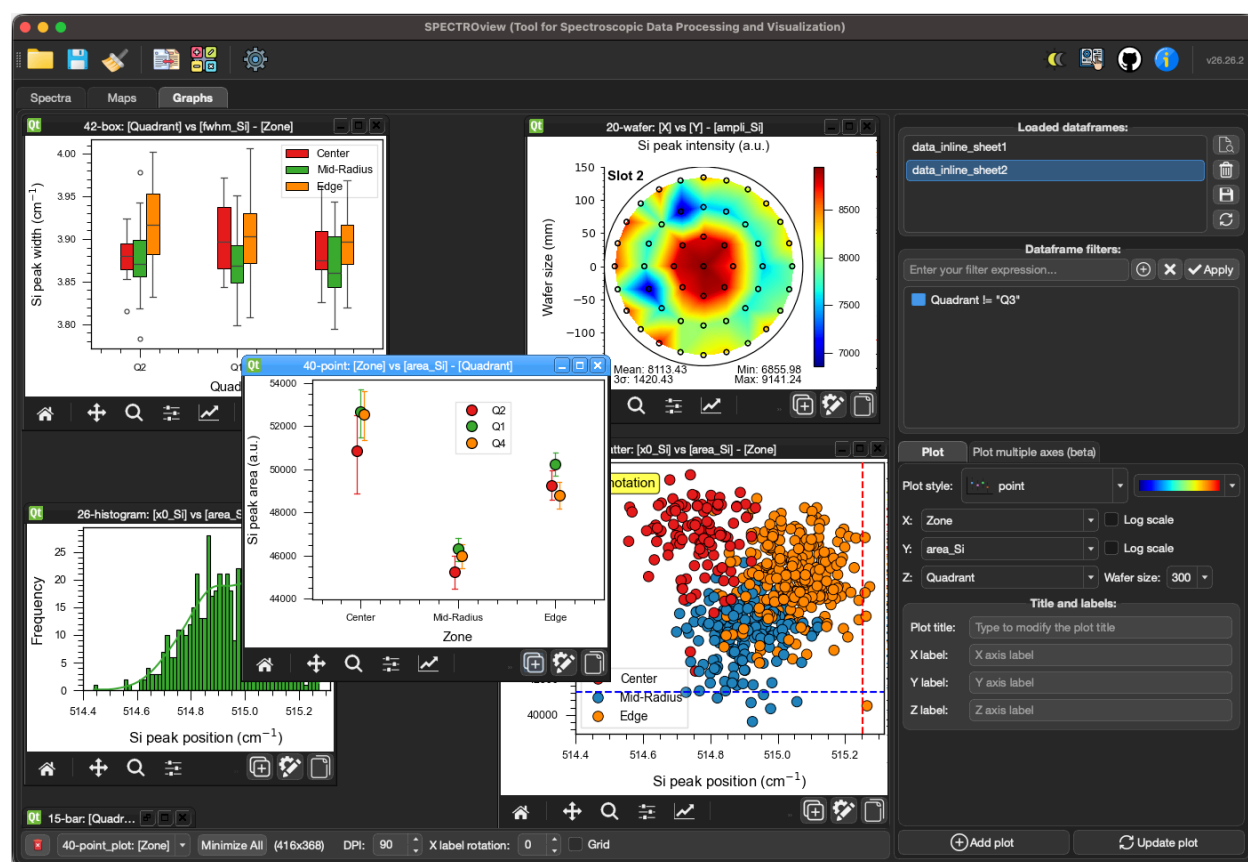


Figure 6: User interface of the Graphs workspace. This workspace provides interactive tools for data selection, filtering, grouping, and visualization through publication-quality statistical plots, including scatter plots, bar charts, box plots, histograms, heatmaps, wafer plots, and trend plots. Generated plots are displayed in independent, resizable, and movable windows that can be customized as needed.

Each plot can be further customized through a dedicated dialog that allows users to adjust axis limits, legend properties (labels, colors, markers), annotations (e.g., draggable vertical/horizontal reference lines, text labels), along with other visual settings. As previously shown, Figure 5 provides an example of a wafer plot displaying fitted parameters together with automatically computed statistics (mean, max, min, and standard deviation). Other examples are shown in Figure 6.

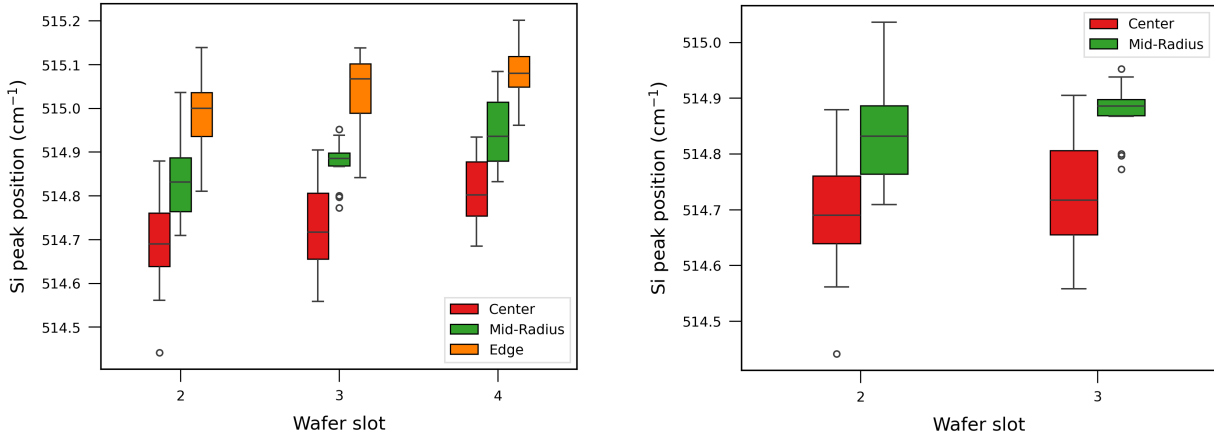


Figure 7: (a) Box plot generated directly from fitting results transferred from the Maps workspace, comparing the fitted silicon Raman peak position for multiple wafers. The data are automatically grouped according to three predefined wafer regions (Center, Mid radius, Edge). (b) The same dataset after applying dynamic filters to exclude Wafer 4 and the Edge region, demonstrating the interactive data exploration capabilities of the Graphs workspace.

Figure 7a illustrates a box-plot comparison of the fitted silicon peak position across three wafers. The Graphs workspace provides a dynamic filtering feature (see the right-side panel in Figure 6) that enables users to subset the displayed data by applying filtering expressions directly to selected DataFrame columns, without modifying the original dataset. For example, Figure 7b was generated from the same dataset as Figure 7a after applying the filters *Slot != "4"* and *Zone != "Edge"* to the *Slot* and *Zone* columns, respectively. These filters exclude measurements from wafer slot 4 and the wafer edge region, illustrating how users can rapidly explore subsets of the data. Additional examples are available in the Graph workspace section within the User Manual,; https://cea-metrocarac.github.io/SPECTROview/user_manual/graphs/.

4. Impact

4.1. Comparison with some existing tools

Table 2 present a comparison of SPECTROview with some existing spectroscopic fitting tools. SPECTROview distinguishes itself along two axes:

First, it is the only open-source tool that integrates the complete workflow from multi-format file support (Table 1) through interactive peak fitting to publication-quality statistical plotting within a single GUI application. Its ergonomic and highly optimized interface is designed to be accessible to a broad audience, including researchers, engineers, and students. It provides an efficient platform for spectral data inspection, as well as the interactive processing and analysis of both small and large-scale hyperspectral datasets, all without requiring the user to write code and, notably, free of any licensing barriers.

Table 2: Compares SPECTROview with representative open-source and proprietary tools for spectroscopic data analysis. NA = not applicable (library without fitting or proprietary implementation); ✓* = partial support ((fityk [1] uses analytical derivatives for some built-in functions).

Feature	SPECTROview	Fitspy [5]	Fityk [1]	RamanSPy [2]	PyFasma [3]	Wire/Labspec
Open source	✓	✓	✓	✓	✓	✗

Cross-platform	✓	✓	✓	✓	✓	✗
GUI for interactive fitting	✓	✓	✓	✗	✗	✓
Hyperspectral map support	✓	✓	✗	✓	✗	✓
Wafer map support	✓	✗	✗	✗	✗	✗
Batched vectorized fitting	✓	✗	✗	✗	✗	✓
Analytical Jacobians	✓	✗	✓*	NA	✗	NA
Integrated statistical plotting	✓	✗	✗	✗	✗	✗
Inter-peaks parameter expression	✓	✓	✓	✗	✓	✗
MVA analysis	✓	✗	✗	✓	✓	✓

Second, its VBF engine introduces a high-performance computational approach that is, to the best of our knowledge, unique among open-source spectroscopy software. To appreciate this contribution, it is instructive to contrast the engine against existing computational paradigms:

- Existing spectroscopy tools (lmfit [4], fitspy [5], fityk [1], PyFasma [3], RamanSPy [2]): Existing open-source curve-fitting tools rely on sequential, per-spectrum optimization loops. Whether implemented via Python iteration (fitspy [5], PyFasma [3]) or C++ optimization (fityk [1]), they optimize one spectrum at a time. None of them formulate the fitting of N spectra as a single batched least-squares problem solved through vectorized array operations.
- Deep learning frameworks (PyTorch [19], TensorFlow [20]): While batched Levenberg-Marquardt implementations exist in machine learning repositories, they rely on automatic differentiation (autograd) designed for neural network training. They are not tailored for evaluating closed-form spectroscopic line shapes (e.g., Gaussian, Lorentzian) or for managing independent per-spectrum convergence, where each spectrum may converge at a different iteration.

The VBF engine bridges this methodological gap by adapting the concept of batched optimization into a lightweight, pure-NumPy architecture. Rather than iterating over spectra in a Python loop, the engine assembles the N Jacobian matrices and normal-equation systems as three-dimensional arrays and solves them in a single vectorized LAPACK [15] dispatch. Analytical Jacobians (derived in closed form for each supported line shape) replace the numerical finite-difference approximations used by most tools, further reducing computational cost. A per-spectrum convergence mask progressively excludes converged spectra from subsequent iterations, ensuring that early convergers incur near-zero computational overhead.

Table 3 presents comparative benchmarking results on four representative datasets performed on a laptop with Intel Core i5-8365U and 16 GB of RAM. The VBF engine achieves speedups of 16× to 62× over the sequential lmfit-based approach depending on dataset size and fit model complexity. Overall, memory consumption is not a limiting factor: even for the largest tested

dataset (16 384 spectra), peak memory usage reaches only 845 MB for less than 1 seconds before instantly dropping back to 80 MB once the convergence completes, well within the capacity of standard workstations or laptops typically equipped with 8-16 GB of RAM. A detailed discussion of the VBF engine's memory scaling behavior is provided in Appendix B.

Table 3 : Benchmarking results on four representative datasets, tested on a PC equipped with an Intel Core i5-8365U CPU and 16 GB of RAM. Fitting time is reported as mean $\pm$ standard deviation over 10 repeated runs to quantify run-to-run variability. The sequential baseline uses the Imfit per-spectrum approach.

	MoS ₂ flake	Array of pixel	4 wafer maps	CL map
Map size(number of pixels)	1520	3721	196	16384
Points per spectrum after cropping	108	71	275	304
Number of peaks	3	1	6	1
Memory usage	5.4 MB	4.4 MB	2.2 MB	80 MB
Peak Memory usage upon fitting	46.7 MB	46.3 MB	18.8 MB	844.9 MB
Fitting time (mean $\pm$ std) - VBF	1.3 $\pm$ 0.1 s	0.5 $\pm$ 0.1 s	1.9 $\pm$ 0.1 s	5.3 $\pm$ 0.3 s
Fitting time (mean $\pm$ std) - other*	22 $\pm$ 1s	31 $\pm$ 2 s	33 $\pm$ 2 s	192 $\pm$ 7 s
Performance boost	16×	62×	17×	36×

* Imfit sequential per-spectrum approach.

4.2 Reproducibility and availability

SPECTROview is released under the GNU General Public License v3.0 and hosted on GitHub at <https://github.com/CEA-MetroCarac/SPECTROview>, where the source code and example datasets are available. The software is also distributed via PyPI and can be installed or updated using a single command:

- `pip install spectroview`
- `pip install --upgrade spectroview`

(In systems where both Python 2 and Python 3 are installed, `pip3 install spectroview` should be used instead)

Comprehensive documentation is available at <https://CEA-MetroCarac.github.io/SPECTROview/> which includes a “User Manual” for standard users and a “Developer Guide” for those wishing to contribute to project.

SPECTROview is currently deployed within the Platform for Nanocharacterisation (PFNC) laboratories and the cleanrooms of CEA-LETI (Grenoble, France) for routine characterization and metrology of advanced semiconductor materials via Raman spectroscopy and/or photoluminescence [21], [22], [23], [24]. It is now fully available to the broader scientific community.

5. Conclusions

SPECTROview provides an integrated, open-source solution for spectroscopic data processing, curve fitting, and visualization. By combining an interactive, highly optimized and ergonomic GUI with a high-performance batched fitting engine (VBF), it addresses a practical gap in the open-source tooling available to researchers working with spectroscopic and hyperspectral datasets. The underlying MVVM software architecture, workspace persistence, and extensible peak-model

registry facilitate both daily laboratory workflows and long-term research reproducibility. While SPECTROview was developed and validated primarily for semiconductor material characterization and metrology, particularly Raman spectroscopy, FTIR and photoluminescence, its core functionalities (including data import, preprocessing, multi-peak fitting, and visualization) are application-independent and can be applied to spectroscopic data from a wide range of disciplines. The software is intended for researchers, engineers, and students working in fields such as materials science, biomedical spectroscopy, chemistry, and geosciences, and users from other application domains are encouraged to evaluate and adapt it to their specific needs.

Despite these strengths, several limitations should be noted. First, the VBF engine applies a single fit model to all spectra in a batch. While no hard-coded limit exists on the number of peaks or line-shape combinations, fitting spectra with many overlapping peaks (e.g., >20–30) becomes increasingly sensitive to initial parameter estimates and constraints. For heterogeneous datasets (such as those from composite or lamellar structures) users should define a comprehensive and universal fit model containing all peaks potentially present across the mapped area, with appropriate bounds and inter-peak relations (if any). Where a given peak is absent, [the VBF engine's noise-aware](#) masking automatically suppresses its amplitude to zero, allowing fitting of the remaining peaks to continue until convergence. Well-defined models with tight constraints also improve both fitting speed and robustness. Second, the multivariate analysis module is limited to PCA and NMF; advanced chemometric methods (MCR-ALS, PLS, ICA) and AI/ML-based features (automatic peak detection, spectral classification) are not yet available.

Future development will prioritize addressing these limitations. Planned directions include GPU-accelerated tensor fitting (via CuPy or JAX) for very large hyperspectral datasets, expanded file format support (including HDF5 and proprietary vendor formats), advanced chemometric methods (MCR-ALS, PLS, ICA) in the [multivariate analysis module](#), a plugin architecture for user-defined peak models within the VBF pipeline, AI/ML-assisted features such as automatic peak detection and intelligent parameter initialization, and a Python scripting API for programmatic batch processing and integration with external workflows. As an open-source project, SPECTROview will continue to evolve through community contributions, which are welcomed via the [GitHub repository](#).

CRedit authorship contribution statement

- **Van Hoan Le:** Writing original draft, Writing review & editing, Testing, Validation, Conceptualization, Software.

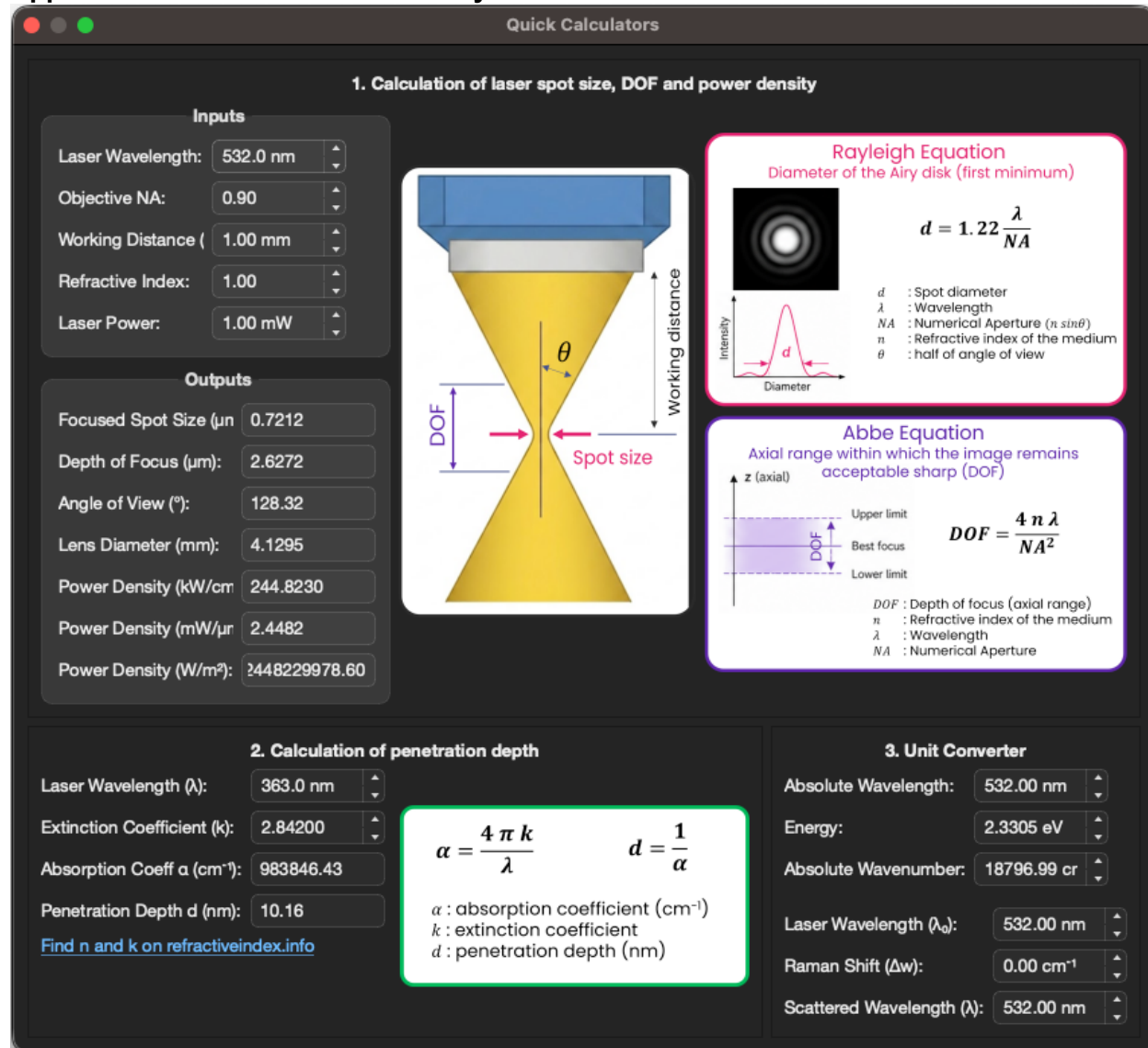
Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work, carried out on the Platform for Nanocharacterisation (PFNC), was supported by the "Recherche Technologique de Base" and "France 2030 - ANR-22-PEEL-0014" programs of the French National Research Agency (ANR). The authors gratefully acknowledge Patrick Quéméré for his assistance during the initial phase of the project.

Appendix A: Quick Calculators Utility



Listing 1: User interface of Quick Calculators Utility. More detailed on this feature at Github documentation site: [here](#).

Appendix B: VBF Engine Memory Scaling

Because the VBF algorithm optimizes all spectra simultaneously in a single mathematical dispatch, its memory footprint is dominated by the construction of a three-dimensional Jacobian tensor of shape ($N \times M \times K$), where N is the number of spectra, M is the number of spectral points per spectrum (after cropping), and K is the number of free fitting parameters. The memory scaling is therefore strictly proportional to ($N \times M \times K$).

For the largest dataset tested in this manuscript (CL map: $N = 16,384$, $M = 304$, $K = 3$), the base memory required to store the preprocessed data matrices is approximately 80 MB. During

simultaneous evaluation of the Jacobian matrices, peak memory allocation reaches 845 MB before dropping back to baseline once convergence completes. These requirements remain well within the capacity of standard workstations or laptops (typically 8-16 GB of RAM). This confirms that while the fully vectorized strategy deliberately trades memory for speed achieving speedups of up to 60× (tested on a laptop with Intel Core i5-8365U) its memory footprint scales predictably and reliably support routine laboratory hyperspectral datasets without requiring out-of-core computing architectures.

References

- [1] M. Wojdyr, "Fityk: a general-purpose peak fitting program," *J. Appl. Crystallogr.*, vol. 43, no. 5, pp. 1126–1128, Oct. 2010, doi: 10.1107/S0021889810030499.
- [2] D. Georgiev, S. V. Pedersen, R. Xie, Á. Fernández-Galiana, M. M. Stevens, and M. Barahona, "RamanSPy: An Open-Source Python Package for Integrative Raman Spectroscopy Data Analysis," *Anal. Chem.*, vol. 96, no. 21, pp. 8492–8500, May 2024, doi: 10.1021/acs.analchem.4c00383.
- [3] E. Pavlou and N. Kourkoumelis, "PyFasma: an open-source, modular Python package for preprocessing and multivariate analysis of Raman spectroscopy data," *Analyst*, vol. 150, no. 14, pp. 3112–3122, 2025, doi: 10.1039/D5AN00452G.
- [4] M. Newville, T. Stensitzki, D. B. Allen, and A. Ingargiola, "LMFIT: Non-Linear Least-Square Minimization and Curve-Fitting for Python," *Zenodo*, Sep. 2014, doi: 10.5281/zenodo.11813.
- [5] P. Quéméré, "Fitspy: A Python package for spectraldecomposition," *J. Open Source Softw.*, vol. 9, no. 96, p. 5868, Apr. 2024, doi: 10.21105/joss.05868.
- [6] R. A. Viscarra Rossel, "ParLeS: Software for chemometric analysis of spectroscopic data," *Chemom. Intell. Lab. Syst.*, vol. 90, no. 1, pp. 72–83, Jan. 2008, doi: 10.1016/j.chemolab.2007.06.006.
- [7] L. A. Reisner, A. Cao, and A. K. Pandya, "An integrated software system for processing, analyzing, and classifying Raman spectra," *Chemom. Intell. Lab. Syst.*, vol. 105, no. 1, pp. 83–90, Jan. 2011, doi: 10.1016/j.chemolab.2010.09.011.
- [8] D. Storozhuk, O. Ryabchykov, J. Popp, and T. Bocklitz, "RAMANMETRIX: a delightful way to analyze Raman spectra," 2022, *arXiv*. doi: 10.48550/ARXIV.2201.07586.
- [9] A. Guzzetti, L. Pauly, A. Hamidi, P. C. Cattin, and F. Canbaz, "RAMBO: an open-source web application for Raman spectral analysis and biological tissue characterization," in *Biomedical Vibrational Spectroscopy 2026: Advances in Research and Industry*, Z. Huang, Ed., San Francisco, United States: SPIE, Mar. 2026, p. 25. doi: 10.1117/12.3080893.
- [10] L. R. R. L. De Souza, C. A. Rios, and M. C. Breitzkreitz, "Raman mapping and Chemometrics: An open access Python-based routine to preprocess and generate chemical maps applying CLS, PCA and PLS methods," *Chemom. Intell. Lab. Syst.*, vol. 269, p. 105616, Feb. 2026, doi: 10.1016/j.chemolab.2025.105616.
- [11] The Qt Company, "Qt for Python (PySide6)." 2024. [Online]. Available: <https://doc.qt.io/qtforpython-6/>
- [12] D. Erb, "pybaselines: A Python library of algorithms for the baseline correction of experimental data," *Zenodo*, Oct. 2022, doi: 10.5281/zenodo.7255880.
- [13] P. Virtanen *et al.*, "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nat. Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020, doi: 10.1038/s41592-019-0686-2.
- [14] D. W. Marquardt, "An Algorithm for Least-Squares Estimation of Nonlinear Parameters," *J. Soc. Ind. Appl. Math.*, vol. 11, no. 2, pp. 431–441, Jun. 1963, doi: 10.1137/0111030.

- [15] E. Anderson *et al.*, *LAPACK Users' Guide*, Third. Society for Industrial and Applied Mathematics, 1999. doi: 10.1137/1.9780898719604.
- [16] M. Waskom, "seaborn: statistical data visualization," *J. Open Source Softw.*, vol. 6, no. 60, p. 3021, Apr. 2021, doi: 10.21105/joss.03021.
- [17] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.
- [18] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, no. 6755, pp. 788–791, Oct. 1999, doi: 10.1038/44565.
- [19] A. Paszke *et al.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," 2019.
- [20] M. Abadi *et al.*, "TensorFlow: A System for Large-Scale Machine Learning," presented at the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), 2016, pp. 265–283. Accessed: Jun. 28, 2026. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>
- [21] Z. Szekrényes *et al.*, "In-line Raman spectroscopy for semiconductors strain engineering and control : Topic/Category AM: Advanced Metrology," in *2025 36th Annual SEMI Advanced Semiconductor Manufacturing Conference (ASMC)*, May 2025, pp. 1–4. doi: 10.1109/ASMC64512.2025.11010779.
- [22] N.-P. Tran *et al.*, "Toward full relaxation of sSOI substrates for PFET device fabrication," *Solid-State Electron.*, vol. 229, p. 109196, Nov. 2025, doi: 10.1016/j.sse.2025.109196.
- [23] D. Barge *et al.*, "300 Mm sSOI engineering with ultra thin buried oxide," *Solid-State Electron.*, vol. 232, p. 109307, Feb. 2026, doi: 10.1016/j.sse.2025.109307.
- [24] D. Monteil, "Simulation of interference-enhanced Raman amplification for advanced microelectronic applications," 2026.